

Event-Driven Programming



Mr. E. O. Tugbeh

AI Product Engineer, HWP Labs.

About Me

- ❖ Bachelor's degree in Mathematics & Computer Science
- ❖ Master's degree in Computer Science (Advanced Software Engineering)
- ❖ Over 12+ years of experience in **Enterprise Software Development & Training**
- ❖ Worked across critical industries such as Banking, Education, Government & Healthcare
- ❖ Member of Computer Professionals (Registration Council of Nigeria) [CPN]
- ❖ Member of Google Developer Group (GDG) & Student Ambassador 2013/2014
- ❖ Microsoft Certified Azure Developer Associate (AZ-204)
- ❖ Claude Certified Solutions Architect & Discord Server Community Mentor



Overview

0

**Role of
JavaScript**

1

**JavaScript
Fundamentals**

2

**BOM BOM
& DOM**

3

**Event
Triggers**

4

**Vibe Coding
w/Claude AI**

Prerequisites

0

**HTML/CSS
Fundamentals**

1

**Charged
Laptop**

2

**Internet
Connection**



A large, white, serif capital letter 'O' is positioned on the left side of the slide. It is set against a background of orange dots on a darker orange field.

Role of JavaScript

Webpage Interactivity

“

JavaScript

An interpreted, high-level programming language used to develop interactive webpages.

Trivia

- ❑ **1995** - JavaScript was created by Brendan Eich (Netscape, *within 10 days*).
- ❑ **2009** - Node.js was created by Ryan Dahl as a JS runtime environment to execute JS outside web browsers.
- ❑ **2011** - React.js was created by Jordan Walke (Facebook) as an open-source JS library for building single-page applications (SPAs).

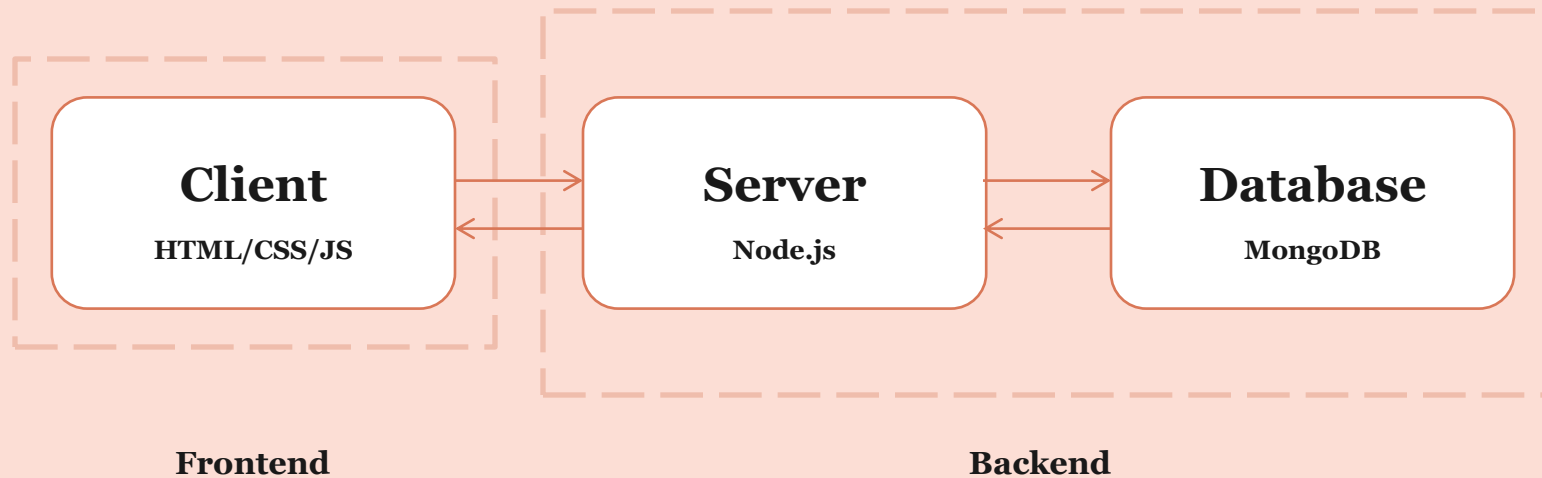


Application Areas

- ❑ Manipulate HTML & CSS (DOM Elements)
- ❑ Handle Webpage Interactions
- ❑ Develop APIs/Web Services (Node.js)
- ❑ Build Desktop Apps (Electron)
- ❑ Build Mobile Apps (React Native)
- ❑ Interface with ETH Smart Contracts (Web3.js)



Web App Architecture





Questions

1

JavaScript Fundamentals

“

Syntax

Set of rules that define the grammatical structure of a programming language.

“

Semantics

The logical meaning and intended behavior behind code constructs.

Statement

- ❑ **Variables & Constants** – Named storage location in memory that can/cannot be changed after declaration (var/let, const).
- ❑ **Data Types** – Defines the type of value to be stored in memory, how much memory it consumes, and what type of operations can be performed on it;
 - ❑ Primitives (Number, String, Boolean, null, undefined)
 - ❑ Non-Primitives (Array, Object, Date)
- ❑ **Operators** – Symbols used to perform computations;
 - ❑ Arithmetic & Assignment
 - ❑ Comparison, Logical & *Ternary

Variables & Constants

1.1

Example 1

1. `// TODO: Compute Student Grade`
2. `var score = 47;`
3. `const total = 60;`
4. `const grade = (score * 100) / total; // 78.33%`
5. `const passed = score >= (total / 2); // true`
6. `const remarks = passed ? "Pass" : "Fail";`
7. `console.log(grade, passed, remarks);`

Identifiers

1. The name given to a variable is known as an **Identifier**.
2. Identifiers can begin with or contain special characters like \$ and _.
3. Numbers cannot be used as the first character of an Identifier.
4. The JavaScript convention for Identifiers is ***camelCase**.
5. Identifiers should be as descriptive as possible. This helps with readability.

Example 2

1. `// TODO: Compute Person Age`
2. `var birthYear = 1952; // BAT`
3. `const thisYear = new Date().getFullYear(); // 2026`
4. `const age = thisYear - birthYear; // 74`
5. `const canVote = age >= 18; // true`
6. `const remarks = canVote ? "Get Your PVC!" : "Pray For Nigeria!";`
7. `console.log(thisYear, age, canVote, remarks);`

Comments

- ❑ Comments are primarily used to **document** code snippets.
- ❑ Comments can also be used to ignore code snippets during **debugging**.
- ❑ Inline comments begin with `// ...`
- ❑ Multi-line comments are wrapped within `/* ... */`



Questions

Data Types

1.2

String Data Type

<code>const x = "Hello, World!";</code>	Creates a string x in memory with assigned value "Hello, World!"
<code>typeof x === "string"</code>	Return true if x is a string data type // true
<code>x.length</code>	Returns the size of string x // 13
<code>x[0]; x[x.length - 1]</code>	Returns the first and last character of string x // H; !
<code>x.at(0); x.at(-1)</code>	Also returns the first and last character of string x (preferred) // H; !
<code>x + "!!"; ` \${x} !!`</code>	Returns the concatenated strings // Hello, World!!!
<code>x.trim()</code>	Removes whitespaces (if any) from the left and right sides of string x
<code>x.toLowerCase(); x.toUpperCase()</code>	Returns string x as lowercase or UPPERCASE
<code>x.includes("u");</code> <code>x.startsWith("h"); x.endsWith("d");</code>	Returns true if string x contains the provided characters // false Returns true if string x starts with or ends with the provided characters // false, false
<code>x.replace("World", "Nigeria");</code> <code>x.replaceAll("o", "@")</code>	Replaces characters in string x // Hello, Nigeria!; Hell@, W@rld!
<code>x.slice(3); x.slice(-6); x.slice(1,5)</code>	Extracts parts of string x using its index // lo, World!; World!; ello
<code>new RegExp("hell", "i").test(x)</code>	Returns true if string x contains the provided characters (case-insensitive)
<code>x.split(",")</code>	Returns string x as an array based on any provided delimiter string // ["Hello", "World!"]

Number Data Type

<code>const x = 1970; const y = 19.70;</code>	Creates a number x in memory with assigned value 1970 Creates a decimal number y in memory with assigned value 19.70
<code>typeof x === "number"</code>	Return true if x is a number data type // true
<code>isNaN(x)</code>	Returns true if x <u>Is Not A Number</u> // false
<code>!isNaN(x)</code>	Returns the negation of true if x <u>Is Not A Number</u> // true
<code>Number(x);</code>	Converts x to a number data type
<code>parseInt(y);</code>	Converts decimal number y to a whole number // 19
<code>parseFloat(y)</code>	Converts decimal number y to a decimal number // 19.7
<code>x.toFixed(2)</code>	Returns number x as a decimal number with 2 decimal places // 1970.00
<code>y.toPrecision(2)</code>	Truncates a decimal number y to exactly 2 digits in size // 19
<code>x.toString(); String(y)</code>	Returns x and y as a strings // "1970"; "19.7"

Array (*List)

- ❑ An Array is an ordered group of values known as **elements**.
- ❑ Arrays can store elements of different data types (numbers, strings, booleans, etc).
- ❑ Array elements are accessed based on their **index** number, i.e. real position minus 1
- ❑ The first element in an Array is located at **x[0]**.
- ❑ The last element in an Array is located at **x[n-1]**, where n is the Array size.
- ❑ In JavaScript, the size of an Array **x.length**, can change as elements are added or removed.
- ❑ **x.toString()** returns Array values as Comma-Separated Values (CSV).
- ❑ **x.join(delimiter)** returns Array values as a string, separated by a delimiter.
- ❑ A nested Array is called a **Matrix**, i.e. an Array of Arrays.

Array Example

```
1. //TODO: Process Jamb Scores
2. const jambScores = [56, 42, 55, 60]; // ENG, MTH, PHY, ECO
3. var size = jambScores.length; // 4
4. var english = jambScores[0]; // 56
5. var economics = jambScores[size - 1]; // 60
6. var scored_56 = jambScores.includes(56); // false
7. var scored_76 = jambScores.includes(76); // true
8. var removedLastValue = jambScores.pop(); // [56, 42, 55], 60
9. var newArraySize = jambScores.push(55); // [56, 42, 55, 55], 4
10. var removedFirstValue = jambScores.shift(); // [42, 55, 55], 56
11. var newArraySize = jambScores.unshift(76); // [76, 42, 55, 55], 4
12. var sortedScoresAsc = jambScores.toSorted((a, b) => a - b); // [42, 55, 55, 76];
13. var sortedScoresDesc = jambScores.toSorted((a, b) => b - a); // [76, 55, 55, 42];
14. var totalScore = jambScores.reduce((total, value) => total + value, 0); // 228
15. var minScore = Math.min(...jambScores); // 42
16. var maxScore = Math.max(...jambScores); // 76
17. var verifyDatatype = Array.isArray(jambScores); // true
18. console.log(jambScores, totalScore, sortedScoresAsc, sortedScoresDesc);
```

Array Methods

<code>const x = [2, 3 ,4]</code>	Creates an array x in memory with 3 elements
<code>Array.isArray(x)</code>	Return true if x is an array data type // true
<code>x.length</code>	Returns the size of array x // 3
<code>x[0]</code>	Returns the first element of array x // 2
<code>x[x.length - 1]</code>	Returns the last element of array x // 4
<code>x.includes(7)</code>	Returns true if 7 is an element of array x // false
<code>x.pop()</code>	Removes the last element of array x, and returns the removed element // 4
<code>x.push(1)</code>	Adds an element to the end of array x, and returns the new size of array x // 3
<code>x.shift()</code>	Removes the first element of array x, and returns the removed element // 2
<code>x.unshift(2)</code>	Adds an element to the start of array x, and returns the new size of array x // 3
<code>x.splice(index, 0, 5);</code> <code>x.splice(index, 1)</code>	Adds or removes element(s) from array x at a given index, where 0 is the number of elements to be removed, and 1 is the number of elements to be added.
<code>x.toString()</code>	Returns the values of array x as <u>Comma-Separated Values (CSV)</u> // 2,3,1
<code>x.join("#")</code>	Returns the values of array x as a string, separated by a <u>delimiter</u> // 2#3#1

Array Methods (Cont'd)

<code>const x = [56, 42, 55, 60]</code>	Creates an array x in memory with 3 elements
<code>x.forEach(fn);</code> <code>x.map(fn)</code>	Modifies elements of array x based on the return value of a given function; Returns a new array based on the return value of a given function
<code>x.filter(fn)</code>	Returns elements that satisfy the given condition in a <u>Logical Function (fn)</u> Eg. <code>aboveAverageFn = (score) => score > 50; // [56, 55, 60]</code>
<code>x.findIndex(fn);</code> <code>x.find(fn)</code>	Returns the first index or element that satisfies the given condition in a <u>Logical Function (fn)</u> Eg. <code>belowAverageFn = (score) => score < 50; // index 2, element 42</code>
<code>x.every(fn);</code>	Returns true if every element satisfies the given condition in a <u>Logical Function (fn)</u> Eg. <code>passedAllFn = (score) => score >= 50; // false</code>
<code>x.some(fn)</code>	Returns true if some elements satisfy the given condition in a <u>Logical Function (fn)</u> Eg. <code>passedSomeFn = (score) => score >= 50; // true</code>
<code>x.reduce(fn, 0)</code>	Returns a computed value based on an <u>Arithmetic Function (fn)</u> where 0 is the default value Eg. <code>totalScoreFn = (total, score) => total + score; // 213</code>
<code>x.toSorted(fn)</code>	Returns a sorted list of array x based on a <u>Sort Function (fn)</u> Eg. <code>ascOrderFn = (i, j) => i-j; // [42, 55, 56, 60]</code> <code>descOrderFn = (i, j) => j-i; // [60, 56, 55, 42]</code>

Object (*HashMap)

- ❑ An Object is a group of **key-value** pairs called **properties**.
- ❑ Objects can store properties of different data types (numbers, strings, booleans, etc).
- ❑ Object properties are accessed based on their **key**, either using the “dot-notation” **x.key** (preferred) or the “index-notation” **x[key]** (dynamic).
- ❑ **Object.keys(x)** returns Object keys as an Array, **Object.values(x)** returns Object values as an Array, and **Object.entries(x)** returns Object properties as a Matrix.
- ❑ **JSON.stringify(x)** returns Object properties as a Serialized string.
- ❑ A nested Object is called a **Tree**, i.e. an Object of Objects.
- ❑ An “Array of Objects” is called a **Collection**.

Object Example

```
1. //TODO: Process Jamb Scores
2. const jambScores = { ENG: 56, MAT: 42, PHY: 55, ECO: 60 };
3. var size = Object.keys(jambScores).length; // 4
4. var english = jambScores.ENG; // 56 (dot-notation preferred)
5. var economics = jambScores["ECO"]; // 60 (index-notation dynamic)

6. jambScores.PHY = 71 // {ENG: 56, MAT: 42, PHY: 71, ECO: 60}
7. jambScores.CHE = 35 // {ENG: 56, MAT: 42, PHY: 71, ECO: 60, CHE: 35}
8. delete jambScores.CHE; // {ENG: 56, MAT: 42, PHY: 71, ECO: 60}
9. var wroteChemistry = Object.keys(jambScores).includes("CHE"); // god forbid!
10. var scored_56 = Object.values(jambScores).includes(56); // true
11. var totalScore = Object.values(jambScores).reduce((total, value) => total + value, 0); // 229
12. var minScore = Math.min(...Object.values(jambScores)); // 42
13. var maxScore = Math.max(...Object.values(jambScores)); // 71
14. var verifyDatatype = typeof jambScores === 'object'; // true
15. console.log(jambScores, totalScore, wroteChemistry, verifyDatatype);
```

Object Methods

<code>const x = {APC:2, PDP:3, NDC:4}</code>	Creates an object x in memory with 3 properties
<code>typeof x === "object"</code>	Return true if x is an object data type // true
<code>Object.keys(x).length</code>	Returns the size of object x // 3
<code>x.APC; x["NDC"]</code>	Returns the value of the referenced key in object x // 2; 4
<code>x.NDC = 1;</code> <code>x.LAB = 0</code>	Updates the value of key NDC to 1 in object x; // {APC:2, PDP:3, NDC:1} Adds new property if the referenced key not found // {APC:2, PDP:3, NDC:1, LAB: 0}
<code>delete x.LAB</code>	Removes the referenced key property from object x // {APC:2, PDP:3, NDC:1}
<code>Object.keys(x)</code>	Returns the keys of object x as an array // [APC, PDP, NDC]
<code>Object.values(x)</code>	Returns the values of object x as an array // [2, 3, 4]
<code>Object.entries(x)</code>	Returns the properties of object x as a <u>Matrix</u> (i.e. an array of arrays) // [["APC", 2], ["PDP", 3], ["NDC", 4]]
<code>JSON.stringify(x)</code>	Returns the properties of object x as a <u>Serialized</u> string // "{ \"APC\":2, \"PDP\":3, \"NDC\":1}"

Date Module

- ❑ Date is an in-built JavaScript **module** for processing date and time.
- ❑ A Date instance can be created with **new Date()**, which returns the current browser/server timestamp.
- ❑ A Date instance can be created with **new Date("YYYY-MM-DD")**.
- ❑ A Date instance can be created with **new Date("YYYY-MM-DD HH:MM:SS")**.
- ❑ **new Date().toString()** returns **"Sun Jan 01 1970 00:00:00 GMT+0100 (West Africa Time)"**.
- ❑ **new Date().toISOString()** returns **"1970-01-01T00:00:00.000Z"** (a standard storage format).
- ❑ **Date.now()** returns the total milliseconds since **Unix Epoch Time** (the beginning of time, 13-digits).
- ❑ A nested Date is called a **Calendar**, i.e. a Date of Dates? **Na lie o! 😊**

Date Example

```
1. // TODO: Format datetime as Tue, 20 Oct 2020 | 3:00 PM
2. const dateObj = new Date("2020-10-20 15:00:00"); // #EndSARS Protest
3. const dayOfWeekIndex = dateObj.getDay(); // 0-6 >> Sun-Sat
4. const dayOfMonth = dateObj.getDate(); // 1-31
5. const monthIndex = dateObj.getMonth(); // 0-11 >> Jan-Dec
6. const year = dateObj.getFullYear(); // 2026
7. const hourIndex = dateObj.getHours(); // 0-23
8. const minuteIndex = dateObj.getMinutes(); // 0-59
9.
   const days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
10.  const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
11.  const meridiem = hourIndex < 12 ? "AM" : "PM";
12.  const hours_f = hourIndex % 12 || 12; // 1-12
13.  const minutes_f = minuteIndex.toString().padStart(2, "0"); // 00-59
14.  const date_f = `${days[dayOfWeekIndex]}, ${dayOfMonth} ${months[monthIndex]} ${year}`;
15.  const time_f = `${hours_f}:${minutes_f} ${meridiem}`;
16.  const output = `${date_f} | ${time_f}`;
17.  console.log(dateObj.toString(), output);
```


Date Methods

<code>const x = new Date()</code>	Creates a date instance x with the browser/server current timestamp
<code>const x1 = new Date("1970-01-01"); const x2 = new Date("1970-01-01 00:00:00")</code>	Creates date instances x1 & x2 with their respective datetime values
<code>x instanceof Date</code>	Returns true if x is a Date data type
<code>x.getDay()</code>	Returns the day index of the week // 0-6 >> Sun-Sat
<code>x.getDate()</code>	Returns the day of the month // 1-31
<code>x.getMonth()</code>	Returns the month index of the year // 0-11 >> Jan-Dec
<code>x.getFullYear()</code>	Returns the year of date instance x // YYYY
<code>x.getHours()</code>	Returns the hour index of date instance x in 24-hours format // 0-23
<code>x.getMinutes()</code>	Returns the minute index of date instance x // 0-59
<code>x.getSeconds()</code>	Returns the seconds index of date instance x // 0-59
<code>x.getMilliseconds()</code>	Returns the milliseconds index of date instance x // 0-999
<code>x.toString()</code>	Returns "Sun Jan 01 1970 00:00:00 GMT+0100 (West Africa Time)"
<code>x.toISOString()</code>	Returns "1970-01-01T00:00:00.000Z" a standard datetime storage format
<code>x.getTime(); Date.now()</code>	Returns date instance x total milliseconds or the current browser/server timestamp total milliseconds, since <u>Unix Epoch Time</u> (the beginning of time, 13-digits)

Math Module

<code>const x = Math.PI</code>	Returns the mathematical value of PI 3.141592653589793
<code>Math.round()</code>	Returns number x rounded to the nearest integer // 3
<code>Math.ceil(x)</code>	Returns number x rounded up to the nearest integer // 4
<code>Math.floor(x)</code>	Returns number x rounded down to the nearest integer // 3
<code>Math.sqrt(4)</code>	Returns the square root of a number // 2
<code>Math.cbrt(8)</code>	Returns the cube root of a number // 2
<code>Math.pow(3, 2)</code>	Returns the power of a number // 9 "All power belongs to Jesus"
<code>Math.abs(-3.142)</code>	Returns the absolute/non-negative value of a number // 3.142
<code>Math.trunc(x)</code>	Returns the integer part of a decimal number // 3
<code>Math.random()</code>	Returns a random decimal number between 0 (inclusive) and less than 1 (exclusive) // 0.124
<code>const y = Math.random() * 10</code>	Returns a random decimal number between 0 (inclusive) and less than 10 (exclusive): // 7.595
<code>Math.floor(y + 1)</code>	A random whole number between 1 and 10 (inclusive) // 7
<code>Math.min(...[56,42,55,55]);</code>	Returns the minimum element in an array of numeric data types // 42
<code>Math.max(...[56,42,55,55])</code>	Returns the maximum element in an array of numeric data types // 56



Questions

1.3

Operators

Arithmetic Operators

Operator	Description	Expression (x = 3 & y = 2)	Result
+	Add	x + y	5
-	Subtract	x - y	1
*	Multiply	x * y	6
/	Divide	x / y	1.5
**	Exponent	x ** y	9
%	Modulus	x % y	1
++	Increment	x++	4
--	Decrement	y--	1

Assignment Operators

Operator	Description	Expression	Result (x)
=	Assign	x = 3	3
+=	Add	x += 2	5
-=	Subtract	x -= 2	1
*=	Multiply	x *= 2	6
/=	Divide	x /= 2	1.5
%=	Modulus	x %= 2	1

Comparison Operators

Operator	Description	Expression (x = 3 & y = 2)	Result
==	Equal	x == y	false
!=	Not equal	x != y	true
===	Equivalent	x === "3"	false
!==	Not equivalent	y === "2"	false
<	Less than	x < y	false
>	Greater than	x > y	true
<=	Less than or equal	x <= y	false
>=	Greater than or equal	x >= y	true

Logical Operators

Operator	Description	Expression (x = 3 & y = 2)	Result
&&	AND	x == 3 && y === "2"	false
	OR	x == 3 y === "2"	true
!	NOT	!(x == 3 y === "2")	false
?	Ternary	x===3 ? "Yes" : "No"	Yes
??	Nullish Coalescing	x ?? 2	false
??=	Nullish Assignment	x ??= 2	false



Questions

Control Flow

Order Of Execution

1.4

“

Conditional Statements

Execute a block of code only when
a specified condition evaluates
to true or false.

“if, else” Statement

```
1. // TODO: Evaluate Jamb Score
2. var input = prompt("Enter Jamb Score:");
3. input = parseInt(input);

4. if (input >= 200) {
5.     alert("Passed!");
6. } else {
7.     alert("Failed!");
8. }
```

“else if” Statement

```
1.  // TODO: Evaluate CGPA
2.  var input = prompt("Enter CGPA:");
3.  input = parseFloat(input);

4.  if (input >= 4.5 && input <= 5) {
5.      alert("First Class!");
6.  } else if (input >= 3.5 && input <= 4.49) {
7.      alert("Second Class Upper!");
8.  } else if (input >= 2.5 && input <= 3.49) {
9.      alert("Second Class Lower!");
10. } else if (input >= 1.5 && input <= 2.49) {
11.     alert("Third Class!");
12. } else if (input >= 1 && input <= 1.49) {
13.     alert("Pass!");
14. } else {
15.     alert("Haba!");
16. }
```

“switch” Statement

```
1. // TODO: Language Translator
2. var input = prompt("Enter time of day:");
3. input = input.trim().toLowerCase();

4. switch (input) {
5.     case "morning":
6.         alert("Ẹ kààárò!");
7.         break;
8.     case "afternoon":
9.         alert("Ẹ kààsán!");
10.        break;
11.    case "evening":
12.        alert("Ẹ kùùròlẹ!");
13.        break;
14.    case "night":
15.        alert("Ẹ káalẹ!");
16.        break;
17.    default: // Good day
18.        alert("Ẹ kú ọjó rere!");
19. }
```

“

Repetitive Statements

Execute a block of code for as long as a specified condition is true.

Repetitive Statements are composed of; **Initialization, Condition, Body & Iterator**

“for” loop

1. `// TODO: Compute Even & Odd Numbers`
2. `const even = [];`
3. `const odd = [];`
4. `for (let i = 1; i <= 10; i++) {`
5. `i % 2 ? odd.push(i) : even.push(i);`
6. `}`
7. `console.log(even.toString()); // 2,4,6,8,10`
8. `console.log(odd.toString()); // 1,3,5,7,9`

“for in/for” Loop

1. `// TODO: Iterate Array Indices & Elements`
2. `const jambScores = [56, 42, 55, 55];`
3. `for (let index in jambScores) {`
4. `console.log(index); // 0,1,2,3`
5. `}`
6. `for (let element of jambScores) {`
7. `console.log(element); // 56,42,55,55`
8. `}`

“while” Loop

```
1. // TODO: Compute Even & Odd Numbers
2. const even = [];
3. const odd = [];

4. let i = 1;
5. while (i <= 10) {
6.   i % 2 ? odd.push(i) : even.push(i);
7.   i++;
8. }

9. console.log(even.toString()); // 2,4,6,8,10
10. console.log(odd.toString()); // 1,3,5,7,9
```

“do, while” Loop

1. `// TODO: Compute Even & Odd Numbers`
2. `const even = [];`
3. `const odd = [];`
4. `let i = 1;`
5. `do {`
6. `i % 2 ? odd.push(i) : even.push(i);`
7. `i++;`
8. `} while (i <= 10);`
9. `console.log(even.toString()); // 2,4,6,8,10`
10. `console.log(odd.toString()); // 1,3,5,7,9`



Questions

1.5

Functions

Functions

- ❑ A Function is a **reusable** block of code that performs a specific task or series of actions.
- ❑ The **function** keyword is used to define a Function; followed by the function **Name**, optional **Parameters** in parenthesis, **Body** block, and optional **Return Value** with the **return** keyword.
- ❑ The **fnName()** syntax is used to call or invoke a Function.
- ❑ The actual values passed during a Function Call is known as **arguments**.
- ❑ *A Function that returns an *indeterminate* value is known as an Impure Function. Eg. modifies a global variable, writes to a database/file, calls an external API, logs to console, uses `Math.random()` or `Date.now()`. Such Functions are said to cause Side Effects.

Example 1

```
1.  // TODO: Format Date
2.  function formatDate(dateParam) {
3.      const dateObj = dateParam ? new Date(dateParam) : new Date();
4.      const dayOfWeekIndex = dateObj.getDay(); // 0-6 >> Sun-Sat
5.      const dayOfMonth = dateObj.getDate(); // 1-31
6.      const monthIndex = dateObj.getMonth(); // 0-11 >> Jan-Dec
7.      const year = dateObj.getFullYear(); // YYYY

8.      const days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
9.      const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];

10.     return `${days[dayOfWeekIndex]}, ${dayOfMonth} ${months[monthIndex]} ${year}`;
11. }

12. const today = formatDate(); // Wed, 8 Jul 2026
13. const endSars = formatDate("2020-10-20"); // Tue, 20 Oct 2020
14. console.log(today, endSars);
```

Example 2

```
1. // TODO: Format Time
2. function formatTime(timeParam) {
3.     const dateObj = timeParam ? new Date(timeParam) : new Date();
4.     const hourIndex = dateObj.getHours(); // 0-23
5.     const minuteIndex = dateObj.getMinutes(); // 0-59
6.
7.     const meridiem = hourIndex < 12 ? "AM" : "PM";
8.     const hoursText = hourIndex % 12 || 12; // 1-12
9.     const minutesText = minuteIndex.toString().padStart(2, "0"); // 00-59
10.    return `${hoursText}:${minutesText} ${meridiem}`;
11. }
12. const systemTime = formatTime(); // 10:00 AM
13. const userTime = formatTime("1970-01-01 15:09:00"); // 3:09 PM
14. console.log(systemTime, userTime);
```


Example 3

```
1.  // TODO: Format Number
2.  function formatNumber(val = 0, as = "default") {
3.      var output = "";
4.      switch (as) {
5.          case "currency":
6.              var locales = "en-NG";
7.              var options = {
8.                  style: "currency",
9.                  currency: "NGN",
10.                 minimumFractionDigits: 2,
11.             };
12.             output = Intl.NumberFormat(locales,
13.                 options).format(val);
14.             break;
15.             case "social":
16.                 var locales = "en";
17.                 var options = {
18.                     notation: "compact",
19.                 };
20.                 output = Intl.NumberFormat(locales,
21.                     options).format(val);
22.                 break;
23.                 default:
24.                     output = val.toLocaleString();
25.             }
26.             return output;
27.         }
28.
29.         console.log(formatNumber(1759)); // 1,759
30.         console.log(formatNumber(1759, "currency")); // ₦1,759.00
31.         console.log(formatNumber(1759, "social")); // 1.8k
```



Questions

1.6

Classes (OOP)

Object-Oriented Programming

“

Object-Oriented Programming

A paradigm that models real-world entities by their attributes and characteristics.

The 4 cardinal concepts of OOP are; **Encapsulation, Abstraction, Inheritance & Polymorphism.**

Form & Function

- ❑ A Class is group of related variables (**properties**) and functions (**methods**).
- ❑ A Class typically contains a **constructor** method to initialize Class properties, and several **getter** and **setter** methods to read and write Class properties “securely”.
- ❑ The **class** keyword is used to define a Class.
- ❑ The **new EntityName()** expression is used to create a Class.
- ❑ The **this** keyword is used to access properties and methods within a Class.

Example 1



```
1.  // TODO: Model Person Entity
2.  class Student {
3.    constructor(record) {
4.      this.surname = record?.surname || "";
5.      this.otherNames = record?.otherNames || "";
6.      this.gender = record?.gender || "";
7.      this.matricNumber = record?.matricNumber || "";
8.    }
9.
10.   get fullName() {
11.     return `${this.surname}, ${this.otherNames}`;
12.   }
13.
14.   registerCourse(courseCode) {}
15.   checkResult(courseCode) {}
16.
17.   const record = {
18.     surname: "Akinyuwa",
19.     otherNames: "Susan Busola",
20.     gender: "Female",
21.     matricNumber: "IPH/2018/001",
22.   };
23.   const student = new Student(record);
24.   console.log(student.fullName);
25.   // Akinyuwa, Susan Busola
```

Example 2



```
1. // TODO: Model Animal Entity
2. class Fish {
3.   constructor(records = []) {
4.     this.collection = records;
5.   }
6.   get report() {
7.     var totalOrders = this.collection.length;
8.     totalOrders = `Orders Fulfilled: ${totalOrders}`;
9.
10.    const fn = (tot, val) => tot + val.price * val.qty;
11.    var totalSales = this.collection.reduce(fn, 0);
12.    totalSales = totalSales.toLocaleString();
13.
14.    return `${totalOrders} | Total Sales: N${totalSales}`;
15.  }
16.  add(type, price, qty = 1) {
17.    this.collection.push({ type, price, qty });
18.  }
19.  remove(fishId) {
20.    const index = fishId - 1;
21.    this.collection.splice(index, 1);
22.  }
23.  const fish = new Fish();
24.  fish.add("Titus/Mackerel", 4500);
25.  fish.add("Catfish", 3000);
26.  fish.add("Stockfish/Okporoko", 250, 4); // 1k
27.  fish.add("Dried Asa Fish", 3000);
28.  fish.add("Smoked Catfish", 5000);
29.  fish.remove(3);
30.  console.log(fish.report());
31.  // Orders Fulfilled: 4 | Total Sales: N15,500
```

Example 3



```
1. // TODO: Model Place Entity
2. class Hotel {
3.   constructor(record) {
4.     this.name = record?.name || "";
5.     this.address = record?.address || "";
6.     this.contactInfo = record?.contactInfo || "";
7.     this.priceList = record?.priceList || [];
8.   }
9.   get priceRange() {
10.    const minPrice = Math.min(...this.priceList); // 50000
11.    const maxPrice = Math.max(...this.priceList);
12.    const minPriceText = minPrice.toLocaleString(); //
13.    const maxPriceText = maxPrice.toLocaleString();
14.    return `Between N${minPriceText} - N${maxPriceText}`;
15.  }
16.   registerGuest(guestInfo) {}
17.   checkIn(guestId, roomNumber) {}
18.   checkOut(guestId) {}
19.   placeOrder(guestId, orderInfo) {}
20. }
21. const record = {
22.   name: "Wesley Hotel",
23.   address: "GRA, Benin",
24.   contactInfo: "+2348169960927",
25.   priceList: [50000, 100000, 150000, 200000, 250000],
26. };
27. const hotel = new Hotel(record);
28. console.log(hotel.priceRange);
29. // Between N50,000 - N250,000
```


Example 4



```
1.  // TODO: Model Thing Entity
2.  class POS {
3.      constructor(record) {
4.          this.cardNumber = record?.cardNumber || ""; // 16-digit
5.          this.expiryDate = record?.expiryDate || ""; // MM/YY
6.          this.cvv = record?.cvv || ""; // ***
7.      }

8.      get isValidCard() {
9.          const [MM, YY] = this.expiryDate.split("/");
10.         const expiryDateObj = new Date(`20${YY}-${MM}-01`);
11.         const expiryDateOK = expiryDateObj.getTime() >
Date.now();
12.         const cardNumberOK = this.cardNumber.length === 3;
13.         const cvvOK = this.cvv.length === 3;

14.         return cardNumberOK && expiryDateOK && cvvOK;

15.     }

16.     checkBalance() {}
17.     transfer(accountNo, bankName, amount) {}
18.     withdraw(amount) {}
19.     confirmTransaction(pin) {}
20. }

21. const record = {
22.     cardNumber: "0002 3481 6996 0927",
23.     expiryDate: "09/26",
24.     cvv: "015", // Card Verification Value
25. };

26. const pos = new POS(record);
27. console.log(pos.isValidCard);
28. // true
```

Example 5

```
1. // TODO: Format Number
2. class MoneyFormat {
3.   constructor(val = 0) {
4.     this.val = val;
5.   }
6.   get standard() {
7.     return this.val.toLocaleString();
8.   }
9.   get currency() {
10.    const locales = "en-NG";
11.    const options = {
12.      style: "currency",
13.      currency: "NGN",
14.      minimumFractionDigits: 2,
15.    };
16.    return Intl.NumberFormat(locales,
```

```
17.   }
18.   get social() {
19.     const locales = "en";
20.     const options = {
21.       notation: "compact",
22.     };
23.     return Intl.NumberFormat(locales,
options).format(this.val);
24.   }
25. }
26.
const mf = new MoneyFormat(1759);
27. console.log(
28.   mf.standard, // 1,759
29.   mf.currency, // ₦1,759.00
30.   mf.social // 1.8k
31. );
```



Questions

2

**BOM BOM
& DOM**

“

Browser Object Model (BOM)

Represents the browser window and its components beyond the webpage.

BOM Module

<code>window.open(URL, "_blank")</code>	Opens the specified URL in a new browser tab/window
<code>window.close()</code>	Closes the current browser window
<code>window.moveTo(x, y)</code>	Repositions the browser window along the x, y axis
<code>window.resizeTo(width, height)</code>	Resizes the browser width and height
<code>window.innerWidth; window.innerHeight;</code>	Returns the current browser width and height in real-time
<code>history.back(); history.forward();</code>	Navigates browser history
<code>history.go(-1), history.go(1),</code>	Navigates browser history using numbers
<code>const x = setTimeout(fn, 1000); clearTimeout(x)</code>	Calls a function once after the specified time elapses in milliseconds
<code>const y = setInterval(fn, 1000); clearInterval(y)</code>	Calls a function repeatedly after the specified time elapses in milliseconds

BOM I/O Module

<code>alert("Message sent!")</code>	Displays the browser native notification dialog
<code>confirm("Are you sure?")</code>	Displays the browser native confirmation dialog
<code>prompt("Enter OTP:", "***")</code>	Displays the browser native input dialog with an optional default input value
<code>console.log(...args)</code>	Prints one or more arguments in the browser/server terminal
<code>console.info(msg)</code>	Prints a message with blue indicator in the browser/server terminal
<code>console.warn(msg)</code>	Prints a message with yellow indicator in the browser/server terminal
<code>console.error(msg)</code>	Prints a message with red indicator in the browser/server terminal
<code>console.table(arr);</code> <code>console.table(obj)</code>	Prints an array or object data type as a table in the browser/server terminal
<code>console.clear()</code>	Clears or resets the browser/server terminal

BOM Location Module

<code>location.href</code>	Returns the webpage full URL // <code>https://northwindai.org /podcast</code>
<code>location.protocol</code>	Returns the webpage protocol // <code>https</code>
<code>location.hostname</code>	Returns the webpage domain // <code>northwindai.org</code>
<code>location.port</code>	Returns the webpage port (if any) // <code>3000</code> or <code>8000</code>
<code>location.pathname</code>	Returns the webpage pathname // <code>/podcast</code>
<code>location.search</code>	Returns the webpage query parameters as string // <code>?upcoming=true</code>
<code>location.hash</code>	Returns the webpage fragment // <code>#top</code>
<code>location.assign(URL)</code>	Navigates to the specified URL and updates the browser history
<code>location.replace(URL)</code>	Navigates to the specified URL without updating the browser history
<code>location.reload()</code>	Reloads the webpage

BOM Navigator Module

<code>navigator.onLine</code>	Returns true if the browser/server detects internet connection
<code>navigator.userAgent</code>	Returns the user-agent header sent by the browser to the server
<code>navigator.geolocation.getCurrentPosition(fn)</code>	Returns the browser's current position
<code>navigator.geolocation.watchPosition(fn)</code>	Returns the browser's current position in real-time
<code>navigator.geolocation.clearWatch()</code>	Stops the browser's current position real-time tracking
<code>position.coords.latitude</code>	Returns the browser's current latitude as a decimal number
<code>position.coords.longitude</code>	Returns the browser's current longitude as a decimal number
<code>position.coords.accuracy</code>	Returns the accuracy of the browser's current position

BOM Storage Module

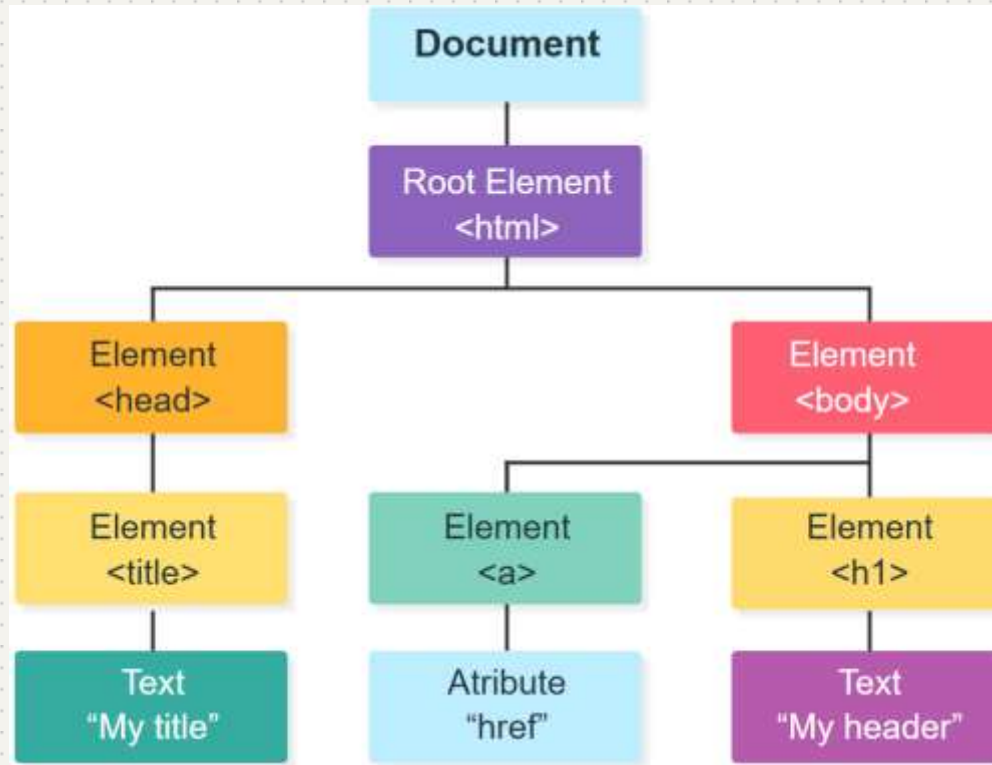
BOM Fetch Module

“

Document Object Model (DOM)

Represents the structure of HTML elements as programmable objects.

A Tree of Nodes



DOM Module

<code>document.title; document.URL;</code>	Read or write the title or full URL of a webpage
<code>document.getElementsByTagName("a")</code>	Returns an array of nodes that match the given tag name
<code>document.getElementsByName("surname")</code>	Returns an array of nodes that match the given name attribute
<code>document.getElementsByClassName("card")</code>	Returns an array of nodes that match the given class attribute
<code>document.getElementById("container")</code>	Returns the first node that match the given id attribute
<code>document.querySelectorAll(".btn")</code>	Returns an array of nodes that match the given selector
<code>document.querySelector("#pri-btn")</code>	Returns the first node that match the given selector
<code>document.forms["login"]["email"].value</code>	Read or write the value of an input field with name/id "email" within a form with name/id "login"

DOM Module (Cont'd)

<code>x.hasAttribute(name)</code>	Returns true is element x has the given attribute name
<code>x.attributeName = value;</code> <code>x.setAttribute(name, value)</code>	Create or update an attribute of element x
<code>x.getAttribute(name)</code>	Read an attribute of element x
<code>x.removeAttribute(name)</code>	Remove an attribute from element x
<code>x.classList.contains("active")</code>	Returns true if the class attribute value of element x contains "active"
<code>x.classList.add("active");</code> <code>x.classList.remove("active")</code>	Add or remove class attribute values of element x
<code>x.classList.toggle("active");</code> <code>x.classList.replace("selected", "not-selected")</code>	Toggle or replace class attribute values of element x
<code>x.style.property</code> (e.g. color, fontSize, padding)	Read or write the style (css) property of element x
<code>x.innerHTML</code> ; <code>x.innerText</code>	Read or write the markup or plain text content of element x
<code>x.click()</code> ; <code>x.focus()</code> ; <code>x.blur()</code> ;	Triggers click, focus or blur event on element x



Questions

3

Event Triggers

“

DOM Events

Enable JavaScript add event listeners or event handlers to HTML elements.

Event Triggers

```
window.addEventListener("load", eventHandlerFn);
```

```
document.getElementById("pri-btn").addEventListener("click", eventHandlerFn)
```

load

click, dblclick, toggle

input, change, select

focus, blur

submit, reset

copy, cut, paste

drag, drop

play, playing, pause, stop

online, offline



Questions

4

**Vibe Coding
w/Claude AI**

> _

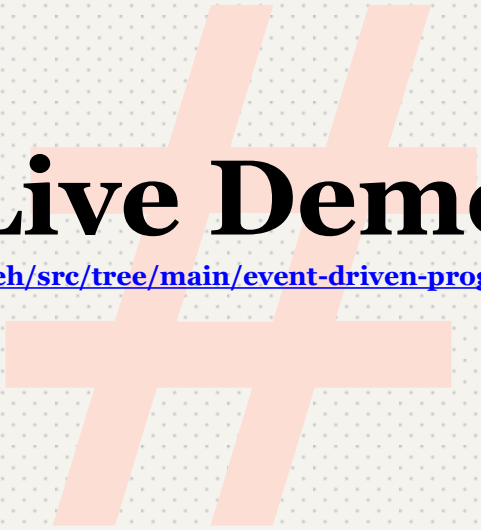
[attach-screenshot]

“generate static html5 semantic tags
and css3 in single file”

Exercise 1

- ❑ **Northwind AI** is an AI Automation and Data Analytics service, provided by HWP Labs.
- ❑ **Northwind AI Podcast** is a platform that promotes tech startups, and application areas of AI within and outside the tech industry.
- ❑ Northwind AI Podcast holds LIVE on X Spaces, on Fridays & Sundays at 8pm (WAT).
- ❑ **Tech Stack:** TypeScript (Next.js), Python (FastAPI), PostgreSQL (Supabase) & Vercel
- ❑ **TODO:** “Vibe code” Northwind AI Podcast homepage using Claude AI, and implement load, input & click events with JavaScript.





Live Demo

<https://github.com/2gbeh/src/tree/main/event-driven-programming/northwind-ai>

Exercise 2

- ❑ **OPay** is a FinTech **super-app** founded in 2018, and acquired by Opera Software.
- ❑ OPay operates across Nigeria, Egypt, Pakistan and other markets offering Mobile Money (payments, transfers, loans, utility bills), ATM Cards, POS Terminals, Ride-hailing, and Food Delivery services.
- ❑ OPay reported serving 45m+ Users, and 1m+ Merchants & Businesses in Nigeria.
- ❑ **Tech Stack:** React Native, Kotlin, Swift, Node.js, PostgreSQL, Redis, Kafka & AWS
- ❑ **TODO:** “Vibe code” OPay App transfer screen using Claude AI, and implement *input* & *click* events with JavaScript.





Live Demo

<https://github.com/2gbeh/src/tree/main/event-driven-programming/opay>

Exercise 3

- ❑ **ExpoCBT** is an open-source CBT **WebApp** developed by E.O. Tugbeh & H.M. Oshomoh.
- ❑ ExpoCBT will be “**built in public**” on **TikTok LIVE**, alongside students of the Dept. of Software Engineering, Obafemi Awolowo University (OAU).
- ❑ Active participants will be added on GitHub as Contributors and assigned Issues.
- ❑ **Tech Stack:** TypeScript (Next.js), PostgreSQL (Supabase) & Vercel
- ❑ **TODO:** “Vibe code” ExpoCBT using Claude AI, implement load, input & click events with JavaScript, and deploy on a Live production env using Vercel.



Live Demo

<https://github.com/hwp-labs/expo-cbt>

NOTE: Create a GitHub account and **[Star]** the repository to be added as a Contributor!



Questions

Thank You!



Resources

1. Uvietobore, T. (2026). [WATCH] SEN106/SEN216 - Introduction to Web Technologies: <https://youtube.com/watch?v=cGoyAmGwDog>
2. Tugbeh, E.O. (2026). [PDF] Event-Driven Programming: <https://github.com/2gbeh/src/blob/main/event-driven-programming/slide.pdf>
3. Tugbeh, E.O. (2026). [ZIP] Source Code Repository: <https://github.com/2gbeh/src/blob/main/event-driven-programming/lab.zip>
4. Tugbeh, E.O. (2026). Code Snippets Markdown File: <https://github.com/2gbeh/src/blob/main/event-driven-programming>
5. OneCompiler JavaScript Online Interpreter - <https://onecompiler.com/javascript>
6. Tugbeh, E.O., Oshomoh, H.M., & Aiyeduyoni S. (2026). Northwind AI Podcast E19 - Statement of Work: Government Subcontractors: <https://x.com/i/spaces/1wGWjjElmNNKQ>
7. Join Northwind AI Community: <https://chat.whatsapp.com/HajjJPJ2HkEorTTITv2FGm>
8. Support Northwind AI: <https://support.northwindai.org>
9. W3Schools – JavaScript Tutorial: <https://w3schools.com/js/default.asp>
10. W3Schools – Introduction to Programming: <https://w3schools.com/programming/index.php>
11. Traversy Media - 12 Important Concepts In The Age Of AI Software Development: https://youtu.be/IJ-FAcYq_o8

Let's Connect

Email	etugbeh@outlook.com
Telephone	(+234) 81 6996 0927
LinkedIn	/in/2gbeh
GitHub	/2gbeh
Twitter/X	/@2gbeh
TikTok	/@2gbeh
Website	https://northwindai.org
Podcast	https://podcast.northwindai.org
Community	https://chat.whatsapp.com/HajjJPJ2HkEorTTITv2FGm